



Tecnun
Universidad
de Navarra

ESCUELA DE INGENIERÍA
INGENIARITZA ESKOLA
SCHOOL OF ENGINEERING

● C++ Laboratory Lessons

Lesson Nº 12

C++
PARA INGENIEROS



Computer Programming II

Prof. Dr. Paul Bustamante / Dr. Jesús Paredes

OUTLINE

1	EXERCISE 1: SAVING TO DISK WITH FORMAT	1
2	EXERCISE 2: ADDING MORE THINGS TO THE FILE.....	2
3	EXERCISE 3: READING MY FILES.....	2
4	EXERCISE 4: FILE READING.	3
5	EXERCISE 5: SIMULATION: A CAPACITOR'S CHARGING CURVE.....	3
6	EXERCISE 6: WRITING STRUCTURES.....	4
7	EXERCISE 7: READING STRUCTURES.....	4
7.1	VARIATION	5
8	EXERCISE 8: READING AND WRITING OBJECTS.....	5

Introduction

In this lesson you are going to do exercises intended to learn how to write data in text files, be able to read them with any text editor, for example *Notepad*, or even read them and make a graph with the data using *Excel*.

1 Exercise 1: Saving to disk with format

This exercise consists of writing a program that keeps asking the user a name and an age, in order to generate a file in which they are stored following the format shown below:

```
Rafael.....38
Juan.....22
Ricardo.....25
Carla.....5
```

The filename of the file in which the data are wanted to be stored is also asked to the user. Next, the source code of the program is shown. You'll have to create the project *Exer1* and the file *DiskOut.cpp*, in which you will copy the following code:

```
// DiscoOut.cpp
#include <fstream>
#include <iostream>
#include <string.h>
using namespace std;
void main()
{
    char name[80], filename[80];
    int age;
    cout << "Name of the file to write:";
    cin >> filename;
    ofstream fout(filename);        //open the file
    if (!fout) { cerr << "Error.."; return; }
    fout.fill('.');
    while(true) {
        cout << "Name(end->exit): ";
        cin >> name;
        if ( strcmp(name, "end") == 0) break;
        cout << "Age: "; cin >> age;

        fout.width(15);
        fout.setf(ios::left);
        fout << name;        //write data in disk
        fout.unsetf(ios::left);
```

```

        fout.width(4);
        fout << age << endl;    // write data in disk
    }
    fout.close();
    cout << "Data stored in: " << filename << endl;
}

```

The generated file can be opened with **Notepad** or Visual Studio itself and you can verify you get the data with the specified format.

2 Exercise 2: Adding more things to the file

Modify the previous program to ask, besides the name and age, for the **address** and the **telephone number** and write them in the file with the following format:

```

Carla.....11..Aizkorri 84.....943215678
Ana.....28..Manuel Lardizabal 15.....943212800

```

Set 30 characters for the address (when writing to file, not for the variable) and 10 characters for the phone. Use a variable of type **long** to store the phone number. For the address, you cannot use **cin**, since it does not accept spaces. You have to use cin's function **getline**: **cin.getline(char*,int n)**:

```

char addr[80];
long phone;
. . .
cout << "Address:";
cin.getline(addr,80);
. . .

```

Before using **getline()**, you must use the function **ignore()** to catch the **enter** that remains stored in the input buffer. You can compile your program and create new files, which can be opened with a text editor.

3 Exercise 3: Reading my files.

This exercise consists of creating a program to read a file stored in the hard disk, whose name has to be asked to the user, and print to the console. To this, you should use the function **getline(..)**, so you can read an entire line.

```

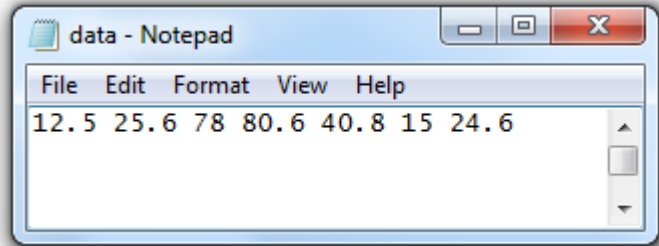
char tmp[120];
char filename[80];
cout << "Name of the file to be read:";
cin >> filename;
fstream fin(filename); //open the file
if (!fin) { cerr << "Error.."; return -1; }
while(true) {
    fin.getline(tmp,120);
    if ( fin.eof() ) break;           //reading ends
    cout << tmp << endl;             //print to console
}
fin.close();

```

Create a project **Exer3** and the file **DiskIn.cpp**, in which you will copy the code above. To run it, you can enter, as the name of the file to read, **DiskIn.cpp** itself.

4 Exercise 4: File reading.

This exercise consists of the following: open the **notepad**, enter a series of numbers and save the file in your working directory with a name that you can remember afterwards (e.g. **data.txt**).



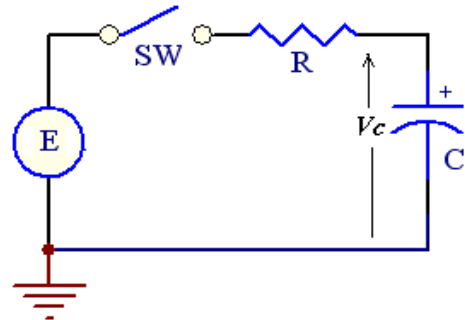
Now you must write a C++ program that asks the **name of the file** to be opened and shows on the screen the following: the numbers in the file, the maximum value and **average value of all** of them.

5 Exercise 5: Simulation: A capacitor's charging curve

In this exercise you will simulate the charging curve of a capacitor, in a conventional RC circuit, as shown in the figure below.

$$\text{The capacitor's charging formula is: } V_c = E * (1 - e^{\frac{-t}{R*C}})$$

Below, the program used to record data from the simulation to a disk file (*.csv) is shown, which can be read by Excel and from which the corresponding graph can be obtained. Files with "*.csv" extension have data separated by "commas".



```
//Writing to *.csv files
#include <fstream>
#include <math.h>
#include <string.h>
#include <iostream>
using namespace std;
void main()
{
    char name[80];
    double R,C,E,dt,Vc=0,t=0;

    cout << "Enter R(KOhm):"; cin >> R;
    cout << "Enter C(uF):"; cin >> C;
    cout << "Enter E(v):"; cin >> E;
    cout << "Enter dt(ms):"; cin >> dt;
    R *= 1e3; C *= 1e-6; dt *= 1e-3;
    cout << "Name of the file (without extension):";
    cin >> name;
    strcat_s(name, ".csv");
    fstream fout(name, ios::out);
    fout << "Values, R:" << R << "C:" << C << "E:" << E << endl;

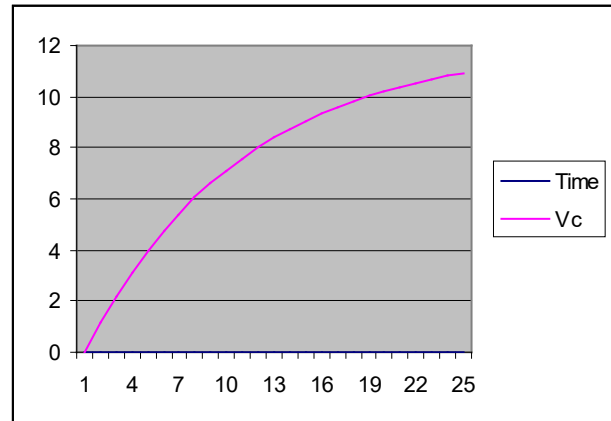
    fout << "Time,Vc" << endl;
    while( Vc < E*0.9) { //10% de E
        Vc = E*(1-exp(-t/(R*C)));
        fout << t << ", " << Vc << endl;
        t += dt;
    }
    fout.close();
    cout << "File " << name << " saved well. Bye " << endl;
}
```

```
}

```

To complete the exercise, you must run Excel and try to load the file you just created and create the graph.

To compile and run the program you must create a project named **Ejer5** and the file **Charge.cpp** to write the source code in it.



6 Exercise 6: Writing structures

Create the following structure:

```
struct task_struct
{
    char name[20];
    double level;
};

```

One way to write to the data of a structure in a file is to do it directly from the main, as shown below:

```
task_struct ts[10];
strcpy_s(ts[0].name, "Welding");
ts[0].level = 8.3;
strcpy_s(ts[1].name, "Fitting");
ts[1].level = 4.3;
strcpy_s(ts[2].name, "Screwing");
ts[2].level = 10.0;
strcpy_s(ts[3].name, "Painting");
ts[3].level = 9.2;
strcpy_s(ts[4].name, "Polishing");
ts[4].level = 5.7;
int count = 5;

ofstream write_object("MyFile.txt");
write_object << ts[0].name << endl << ts[0].level << endl;
write_object.close();

```

As “**write_object**” is an object of class **ofstream**, it can be passed as an input argument to a function. Therefore, you must create a function called “**write_to_file**” in the main whose input arguments are a “**task_struct**” structure and an object of the **ofstream** class. The function header is provided below:

```
void write_to_file(task_struct& , ofstream&);

```

Remove from the above example the part where a **task_struct** structure is written to the file. Now, write the variable “**count**” to the file. Then, run a for loop to call the “**write_to_file**” function and write all the **task_struct** structures within in the “**ts**” array.

7 Exercise 7: Reading structures

Please note that to make the exercise easier, the data from the previous exercise has to be saved without formatting (i.e. without having used the **fill(‘.’)** function of the **ofstream** class).

Structures can also be read from the main because the way the file was written is known: first a number (corresponding to “**count**”), then all the **task_struct** structures. An example is shown below:

```
task_struct tt;
ifstream read_object("MyFile.txt");
int cont;
read_object >> cont;
read_object >> tt.name >> tt.level;
```

In this exercise you are asked to make a function called “**read_task_from_file**” whose only input argument is an object of class `ifstream`, and whose output variable is a `task_struct` structure. Modify the **main** file, to call this function inside a loop, and read all the `task_struct` structures stored in the file.

7.1 Variation

Instead of creating a function that reads the structures one by one, make a function called “**read_task_array_from_file**” that returns the number of structures and a dynamic array with all the structures stored in the file.

8 Exercise 8: Reading and writing objects

Modify exercise 6 and 7 by replacing the `task_struct` structure with the `task_class` class, with all variables `protected`.

Note that you are NOT allowed to use **GetName()** or **GetLevel()** functions (or similar) that return the values of `private/protected` variables. Instead, make a function that belongs to the class, which writes the objects.